

## **OSSP (Operating Systems And Systems Programming) - 25CS2104E**

**Name :** P. Bhavana Manaswini

**Roll No :** 2520030154

**Sec :** 9

### **Week -4**

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

### **Multiple child processes using wait()**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    pid_t pid;
```

```
    for (i = 0; i < 3; i++)
```

```
    {
```

```
        pid = fork();
```

```

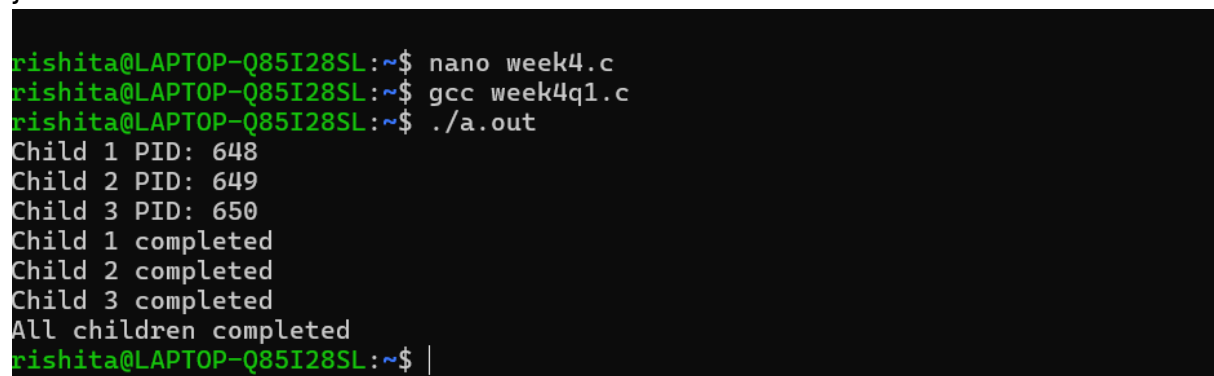
    if (pid == 0)
    {
        printf("Child %d PID: %d\n", i + 1, getpid());
        sleep(i + 1);
        printf("Child %d completed\n", i + 1);
        exit(0);
    }
}

for (i = 0; i < 3; i++)
{
    wait(NULL);
}

printf("All children completed\n");

return 0;
}

```



```

rishita@LAPTOP-Q85I28SL:~$ nano week4.c
rishita@LAPTOP-Q85I28SL:~$ gcc week4q1.c
rishita@LAPTOP-Q85I28SL:~$ ./a.out
Child 1 PID: 648
Child 2 PID: 649
Child 3 PID: 650
Child 1 completed
Child 2 completed
Child 3 completed
All children completed
rishita@LAPTOP-Q85I28SL:~$ |

```

## Multiple child processes using waitpid()

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    pid_t pid[3];
```

```
    for (i = 0; i < 3; i++)
```

```
    {
```

```
        pid[i] = fork();
```

```
        if (pid[i] == 0)
```

```
        {
```

```
            printf("Child %d PID: %d\n", i + 1, getpid());
```

```
            sleep(i + 1);
```

```
            printf("Child %d completed\n", i + 1);
```

```
            exit(0);
```

```
        }
```

```
    }
```

```
    for (i = 0; i < 3; i++)
```

```
    {
```

```
        waitpid(pid[i], NULL, 0);
```

```
        printf("Child PID %d terminated\n", pid[i]);
```

```
    }
```

```
    printf("All children completed\n");
```

```
    return 0;
```

```

}
ishita@LAPTOP-Q85I28SL:~$ nano week4q2.c
ishita@LAPTOP-Q85I28SL:~$ gcc week4q2.c
ishita@LAPTOP-Q85I28SL:~$ ./a.out
child 1 PID: 670
child 2 PID: 671
child 3 PID: 672
child 1 completed
child PID 670 terminated
child 2 completed
child PID 671 terminated
child 3 completed
child PID 672 terminated
all children completed
ishita@LAPTOP-Q85I28SL:~$ |

```

## Zombie process

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if (pid == 0)
```

```
    {
```

```
        printf("Child PID: %d\n", getpid());
```

```
        exit(0);
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("Parent PID: %d\n", getpid());
```

```
        printf("Child PID: %d\n", pid);
```

```
    sleep(5);

    wait(NULL);

    printf("Child process terminated\n");
}

return 0;
}
```

```
rishita@LAPTOP-Q85I28SL:~$ nano week4q3.c
rishita@LAPTOP-Q85I28SL:~$ gcc week4q3.c
rishita@LAPTOP-Q85I28SL:~$ ./a.out
Parent PID: 688
Child PID: 689
Child PID: 689
Child process terminated
rishita@LAPTOP-Q85I28SL:~$ |
```